

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I ~~S. Chandra Shekhar Reddy (192372163)~~ (Name) Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SIMATS ENGINEERING

Assessment Tool 2 — Scenario Based Assignment

Course: Artificial Intelligence (CSA17)

CO2: Search & Game-Solving Techniques — Greedy Search, A*, CSP, Minimax, Alpha-Beta Pruning

Question 1: Emergency Drone Delivery — Greedy Best-First Search vs A* (10 Marks)

Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic — roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. Due to urgency, the drone must quickly decide routes with minimum delay and risk.

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Greedy Best-First Search (GBFS) expands the node that appears closest to the goal purely based on the heuristic value $h(n)$ — the straight-line distance to the destination — while completely ignoring the actual path cost $g(n)$ accumulated so far. In this scenario, the drone would repeatedly fly toward whichever waypoint looks nearest to the delivery point on a straight line, without considering whether the terrain or weather along that route makes it expensive, slow, or unsafe.

Strengths:

- Very fast and computationally light, since it only evaluates $h(n)$ — well suited to the drone's urgency for a quick decision.
- Works reasonably well when the heuristic is highly accurate and the environment is largely static.
- Requires little memory compared to exhaustive search, useful for onboard/limited drone hardware.

Weaknesses:

- Not optimal — it can choose a route that looks close in straight-line distance but is actually long, costly, or hazardous once terrain/weather cost is factored in.
- Not complete in general graphs with cycles/dead-ends — the drone can get trapped chasing a promising-looking but blocked direction.
- Ignores accumulated risk and cost, so it may repeatedly choose paths through dangerous or high-cost flood zones simply because they look geometrically closer to the goal.
- Highly sensitive to heuristic quality; a poor or misleading heuristic (e.g., a straight line over a now-flooded area) directly misleads the search since no cost information corrects it.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

A* evaluates every candidate node using $f(n) = g(n) + h(n)$, combining the real cost already incurred ($g(n)$ — which naturally reflects variable terrain and weather costs) with the estimated remaining distance ($h(n)$). This balances 'how much has this route already cost' against 'how promising does the rest of the route look'. As a result, the drone does not simply chase the geometrically nearest-looking waypoint; it favours the path that is genuinely cheapest overall once cost and estimate are combined. Provided the straight-line-distance heuristic is

admissible (never overestimates true remaining cost, which holds for Euclidean distance versus real terrain-adjusted travel cost), A* guarantees an optimal route, and it will actively avoid a route that looks close but is expensive/risky in practice.

iii. Analyze the impact of heuristic accuracy on both algorithms

For GBFS, heuristic accuracy is everything — since cost is not tracked, an inaccurate heuristic (e.g., ignoring a flooded shortcut) directly leads to a poor or unsafe route with no internal correction mechanism. For A*, heuristic accuracy mainly affects *efficiency* rather than correctness: as long as $h(n)$ remains admissible, A* still finds the optimal path even with a weak heuristic, but it will expand more nodes and take longer. A tighter, more accurate heuristic reduces the number of nodes explored for both algorithms, but A* remains safe and optimal across a wider range of heuristic quality, whereas GBFS's output quality degrades directly with heuristic quality.

iv. Discuss how dynamic changes (blocked paths) affect search performance

Both algorithms, in their classical form, assume a static graph, so a sudden road blockage invalidates part of the previously explored/planned route and forces replanning. GBFS, having no concept of accumulated cost, must effectively restart its greedy chase from the point of disruption with no memory of which alternatives were previously discarded, potentially repeating poor decisions. A* retains richer cost information (g-values already computed for other frontier nodes), so when a path is blocked it can more gracefully fall back on the next-best $f(n)$ node already in its open list, and can be extended into real-time replanning variants (e.g., D* Lite, Anytime Repairing A*) that incrementally repair the search rather than starting over — making it considerably more resilient to dynamic obstacle updates.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

A* (ideally an incremental/real-time variant such as D* Lite or Anytime A*) is the more suitable algorithm for this scenario. Although GBFS is faster per decision, the emergency medicine-delivery context demands both safety and near-optimal routing — a wrong greedy choice could waste critical time or route the drone through a hazardous flooded area. A* guarantees optimality under an admissible heuristic, naturally incorporates variable terrain/weather cost through $g(n)$, and can be adapted with incremental replanning to react to blocked paths in real time. The additional computation cost of A* is an acceptable trade-off against the life-safety and reliability requirements of emergency delivery, whereas GBFS's speed advantage does not outweigh its risk of unsafe or inefficient routes.

Question 2: Smart City Traffic Signal Optimization — Local Search Methods (10 Marks)

Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize total waiting time, fuel consumption, and traffic congestion. The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Formulation. Formulate this as an optimization problem and explain why traditional search is inefficient

This is naturally formulated as a continuous/combinatorial optimization problem rather than a path-finding problem: a **state** is a complete configuration of signal timings (phase durations/offsets) across all intersections; the **objective/cost function** is a weighted combination of total waiting time, fuel consumption, and congestion level; and the goal is to find the configuration that minimizes this cost, not to reach a specific 'goal state'. Traditional systematic search algorithms (BFS, DFS, A*) are designed to find a path to a defined goal in a discrete graph — here there is no single goal state, the branching factor across hundreds of intersections with many possible timing combinations is astronomically large, and the environment changes continuously, making exhaustive or path-based search computationally infeasible. Local search / optimization techniques, which iteratively improve a candidate solution rather than search a path, are far better suited.

i. Explain how Hill Climbing would work in this scenario and identify its limitations

Hill Climbing would start from an initial signal-timing configuration and repeatedly move to a neighbouring configuration (e.g., adjusting one intersection's green-light duration) only if it improves the objective (lower waiting time/fuel/congestion), stopping when no neighbour improves further. It is simple and fast to compute, making it attractive for real-time signal tuning.

Limitations:

- Can easily get trapped in a local optimum — a configuration that is better than all its immediate neighbours but far worse than the true city-wide optimum.
- No backtracking, so a poor early choice cannot be undone.
- Highly dependent on the starting configuration and the definition of 'neighbour'.
- Struggles to keep up with continuously changing traffic patterns since it only ever takes small, greedy local steps.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

- **Local maxima/minima:** A signal-timing pattern that minimizes congestion for one cluster of intersections but leaves the overall city network suboptimal — e.g., optimizing one busy junction while inadvertently creating a bottleneck one block away.
- **Plateaus:** Regions where many neighbouring timing adjustments produce almost identical waiting-time outcomes (e.g., shifting a green phase by a few seconds makes no measurable difference), causing the algorithm to wander without clear improvement signals.
- **Ridges:** Sequences of intersections where the truly optimal joint configuration requires simultaneously adjusting several signals together; moving just one signal at a time (as hill climbing does) may look like it worsens things temporarily even though a coordinated set of changes would produce a much better outcome overall.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Simulated Annealing (SA) introduces controlled randomness: it occasionally accepts a move to a worse configuration with a probability that decreases over time (following a 'temperature' schedule). This lets the search escape local optima and cross plateaus/ridges early on, while gradually converging to a strong solution as the temperature cools — well suited to continuously shifting traffic patterns since it keeps exploring broadly before settling.

Genetic Algorithms (GA) maintain a population of candidate signal-timing plans simultaneously and evolve them over generations using selection (favouring low-cost plans), crossover (combining good partial solutions from different intersections), and mutation (introducing new variation). Because many candidate solutions are explored in parallel across diverse regions of the search space, GAs are naturally resistant to local optima and scale well to the huge combinatorial space of hundreds of intersections; they can also be periodically re-run, seeded with recent good solutions, to adapt to changing traffic conditions.

iv. Recommend the best approach and justify your choice based on scalability and adaptability

A Genetic Algorithm (optionally hybridized with Simulated Annealing / local search refinement) is recommended. Given the scale (hundreds of intersections), the need for continuous adaptation to changing traffic, and the risk of local optima, GA's population-based, parallel exploration scales more naturally across many interacting intersections and can be re-seeded frequently to stay adaptive. Simulated Annealing is a strong lighter-weight alternative for smaller sub-regions or faster embedded controllers. Pure Hill Climbing is not recommended alone due to its vulnerability to local maxima, plateaus, and ridges in this large, dynamic search space.

Question 3: Mars Rover — Online Search in Unknown Terrain (10 Marks)

Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability, and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

i. Explain why this problem requires an online search agent instead of offline search

Offline search algorithms (like standard A*) assume the agent has complete knowledge of the environment/graph before planning begins, computing an entire solution path before taking any action. The Mars rover, however, has no complete map of the unknown terrain in advance — it can only perceive its immediate surroundings through onboard sensors. An **online search agent** interleaves computation and action: it takes an action, observes the actual outcome/environment, updates its knowledge, and then decides the next action — which is the only viable approach when the map is unknown and must be discovered incrementally through exploration.

ii. Analyze the challenges of partial observability and dynamic environments

- **Partial observability:** The rover only senses a limited local area, so hazards like craters or loose rocks just beyond sensor range are unknown until it gets closer, risking unsafe moves based on incomplete information.
- **Dynamic environment:** Martian terrain can shift (dust storms, sand movement, unstable rocks), meaning even previously explored/'known' areas may change, so knowledge can become stale and must be periodically re-validated.
- Together these increase the risk of costly or unsafe backtracking, wasted energy/time, and the need for conservative, safety-aware decision-making since the rover cannot fully trust its existing model of the world.

iii. Describe how the rover builds and updates its internal model

As the rover moves, it uses sensor data (cameras, LIDAR/depth sensors, wheel odometry) to incrementally construct an internal map, marking regions as safe/traversed, known hazards, or still-unknown — similar in spirit to SLAM (Simultaneous Localization and Mapping). Each new observation updates cost/traversability estimates for nearby cells, and the rover revises its plan using this refreshed model (an approach used in algorithms such as Learning Real-Time A*, LRTA*, which updates heuristic estimates for states as they are actually experienced, so the rover 'learns' better estimates of true costs the more it explores).

iv. Compare online search with uninformed and informed search strategies

- **Uninformed search (BFS/DFS):** Explores blindly without any heuristic guidance and assumes the graph is known; wholly impractical here since it wastes effort with no sense of direction and cannot function without a complete map.
- **Informed offline search (A*):** Needs a known map and admissible heuristic computed in advance to search a path to a goal before acting — not directly usable when the environment is unknown.
- **Online search:** Makes decisions step-by-step using only locally available information, updates heuristic/cost estimates from real experience, and interleaves acting with planning — the only category of the three that fits an unknown, partially observable, dynamic environment like Mars terrain.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

An online informed search approach such as Learning Real-Time A* (LRTA*), combined with periodic local replanning, is the most suitable. It preserves the benefit of heuristic guidance (efficient, goal-directed

movement) while remaining safe under uncertainty because decisions are continuously revised as the rover senses and learns more about its surroundings. This balances adaptability (updating the model as terrain is discovered or changes), safety (avoiding committing to long-range plans based on stale or incomplete information), and reasonable efficiency (using learned heuristic estimates rather than blind uninformed search).

Question 4: University Exam Timetabling — Constraint Satisfaction Problem (10 Marks)

Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: no student should have overlapping exams, a limited number of exam halls, certain subjects must be scheduled before others, faculty availability constraints, and special accommodations for some students. Complexity increases as the number of courses and students grows.

Formulation. Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

Variables: Each scheduled exam/subject is a variable (e.g., Exam_DBMS, Exam_AI, Exam_OS, ...).

Domains: For each exam-variable, the domain is the set of feasible (timeslot, exam-hall) pairs it could be assigned — e.g., {(Slot1, Hall A), (Slot1, Hall B), (Slot2, Hall A), ...}.

Constraints:

- *No overlapping student exams (all-different / conflict constraint):* any two exams sharing at least one common student cannot be assigned the same timeslot.
- *Limited exam halls (capacity/resource constraint):* the number of exams held in the same timeslot cannot exceed the number of available halls, and each hall's seating capacity must not be exceeded.
- *Precedence constraints:* certain subjects must be scheduled strictly before others (ordering constraint on timeslot values across specific exam-variable pairs).
- *Faculty availability:* the invigilating/setting faculty assigned to an exam must be free during that exam's timeslot.
- *Special accommodations:* some students require extra time or a separate room, which adds additional room/timeslot constraints tied to specific exam-student pairs (e.g., reserving a distinct slot/hall combination).

ii. Explain how backtracking search works in this scenario

Backtracking search assigns exams to (timeslot, hall) values one variable at a time. After each assignment, it checks consistency against all constraints involving already-assigned exams (no student conflict, hall capacity not exceeded, precedence respected, faculty free). If the assignment is consistent, the algorithm proceeds to the next unassigned exam; if a dead end is reached (no valid value satisfies all constraints), it backtracks to the most recently assigned exam and tries an alternative value from its domain. This continues until either a complete consistent timetable is found or the search space is exhausted.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

Rather than discovering conflicts only after fully assigning a new variable, forward checking immediately removes now-inconsistent values from the domains of related unassigned exams as soon as an assignment is made — e.g., once Exam_A is fixed to Slot1/Hall1, any exam sharing students with Exam_A has Slot1 removed from its domain, and any exam that would exceed Hall1's capacity in Slot1 has that pairing removed. This prunes the search tree early, avoids wasted exploration down branches that are already guaranteed to fail, and can detect an impossible assignment (a domain wiped to empty) far sooner than plain backtracking — substantially improving efficiency, especially as the number of courses and students grows.

iv. Analyze real-world challenges and justify the best strategy for scalability

As the number of courses, students, and constraints grows, the search space becomes combinatorially explosive, some constraints are soft/preferential (e.g., faculty preferences) rather than hard, and requirements may change close to the exam period, requiring fast re-scheduling. Pure backtracking alone does not scale well to large real-world instances.

Recommended strategy: Combine backtracking with constraint propagation techniques (forward checking and/or arc-consistency, e.g., AC-3) to prune domains early, and use good variable/value ordering heuristics — Minimum Remaining Values (MRV) to pick the most-constrained exam next, and Least Constraining Value (LCV) to try the value that rules out the fewest options for others. For very large-scale, frequently-changing timetables, augmenting this with local search repair methods (e.g., min-conflicts) allows the system to quickly patch an existing timetable after late changes rather than re-solving from scratch, giving the best balance of correctness and scalability.

Question 5: Strategic Game AI — Minimax & Alpha-Beta Pruning (10 Marks)

Scenario: A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, and evaluate complex game states with multiple possible moves. The decision tree is extremely large, making computation expensive.

i. Explain how the Minimax algorithm models this problem

Minimax represents the game as a tree in which nodes alternate between the AI's turn (MAX, trying to maximize its outcome) and the opponent's turn (MIN, assumed to play optimally to minimize the AI's outcome). Values are computed at terminal/leaf states (or a cut-off depth) using an evaluation function, then propagated back up the tree: MIN nodes take the minimum of their children's values (worst case for the AI) and MAX nodes take the maximum (the AI's best achievable guarantee). The AI then chooses the move at the root leading to the branch with the highest minimax value — i.e., the best outcome the AI can guarantee assuming a rational, worst-case-optimal opponent.

ii. Analyze the computational challenges of large game trees

- The branching factor in a real-time strategy game is extremely large, since at each turn there may be many possible unit actions/combinations, causing combinatorial explosion.
- Deep look-ahead is often needed for good long-term strategy, but tree size grows exponentially with depth (b^d), making full-depth search computationally infeasible.
- Strict real-time constraints mean the AI has only milliseconds to decide, ruling out exhaustive search to terminal game states.
- Storing/evaluating this many states also has significant memory and processing overhead.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

Alpha-beta pruning augments Minimax with two bounds tracked during traversal: alpha (the best value MAX can already guarantee along the current path) and beta (the best value MIN can already guarantee). While exploring a node's children, if a value is found such that $\alpha \geq \beta$, the remaining children of that node cannot possibly affect the final decision at the root (the opponent, playing optimally, would never allow the game to reach that branch), so the subtree is safely **pruned** without being evaluated at all. This does not change the final minimax value but can, with good move ordering, reduce the effective branching factor from b to roughly $\sqrt{b}$ — allowing the AI to search substantially deeper within the same time budget.

iv. Design an evaluation function and justify the factors considered

Since the tree cannot be searched to terminal states in real time, a heuristic evaluation function estimates the desirability of a non-terminal game state. Relevant factors for an RTS game could include:

- Difference in unit count/health between the AI and opponent (military strength).
- Resource stockpiles and production capacity (economic strength).
- Map/territory control and positional advantage (strategic reach).
- Technology/upgrade level achieved relative to the opponent.
- Proximity of threats to key assets (defensive risk).

Each factor is typically combined as a weighted sum, with weights tuned through testing, expert knowledge, or learning (e.g., self-play tuning), since together they correlate strongly with a player's actual likelihood of eventually winning even though the game has not concluded.

v. Recommend a strategy for balancing optimality and real-time performance

Recommended strategy: Use iterative deepening alpha-beta search — searching depth 1, then 2, then 3, and so on, always keeping the best move found so far, so the AI can be interrupted at any moment by the time limit and still return a reasonable decision. Combine this with strong move-ordering heuristics (e.g., killer moves, history heuristics) to maximize pruning effectiveness, transposition tables to avoid recomputing repeated states, and a well-tuned evaluation function to cut search short at a reasonable depth. This sacrifices guaranteed global optimality (which is computationally impossible under real-time constraints) in favour of a strong, time-bounded, near-optimal decision — the standard and proven approach used in competitive real-time game AI.